

04 行列式

4.4 手解行列式

用余子式与拉普拉斯展开，一步步手算 3×3 方阵的行列式



鸢尾花书·《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>

本节你将掌握的核心技能

1 余子矩阵：去掉指定元素所在行、列后剩下的矩阵

2 余子式：余子矩阵的行列式，是一个标量

3 代数余子式：在余子式基础上做符号调整， $C_{ij} = (-1)^{i+j} M_{ij}$

4 拉普拉斯展开：选定任意一行或列，元素乘以代数余子式并求和，得到行列式

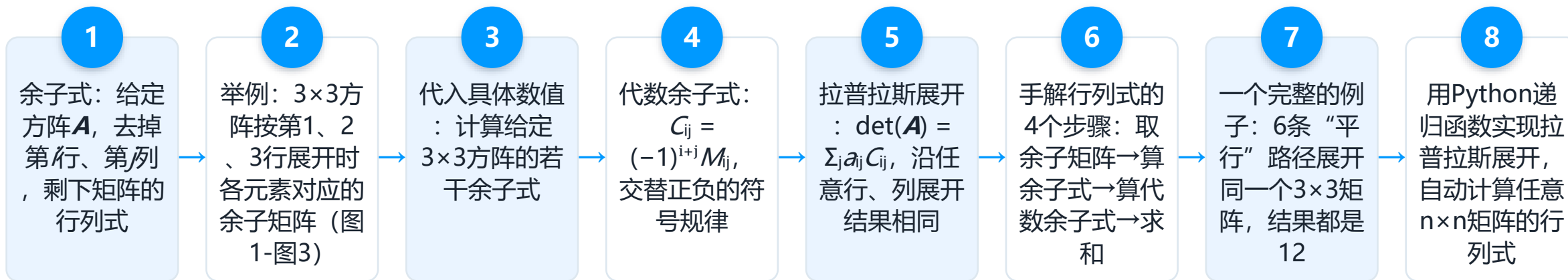
5 优先选择含零最多的行或列展开，可以减少计算量

6 拉普拉斯展开适用于任意 $n \times n$ 方阵，是行列式递归定义的基础

7 用递归的Python函数实现行列式计算，加深对算法结构的理解

8 手算的过程帮助建立对4.3节行列式性质（如共线为0）更扎实的直觉

本节知识脉络



3×3公式不好记，我们需要更结构化的方法

$$\det(A) = a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + \cdots - a_{11}a_{23}a_{32}$$

- 本章前面已经给出3×3矩阵 A 行列式的完整公式，但这个公式项数多、正负号交错，并不容易记住
- 本节介绍一个更具结构化的方法——拉普拉斯展开（Laplace Expansion），并引入余子式（minor）、代数余子式（cofactor）两个概念
- 不需要手算行列式的读者，这一节可以跳过；也可以直接跳到本节末尾的Python代码，用程序完成计算

余子式：从矩阵到标量

$$M_{ij} = \det(\mathbf{A}_{ij}), \mathbf{A}_{ij} \text{ 形状为 } (n-1) \times (n-1)$$

- 给定 $n \times n$ 方阵 $\mathbf{A}$ ，第 i 行、第 j 列元素 a_{ij} 对应的余子式记作 M_{ij}
- 余子式 M_{ij} 是从 $\mathbf{A}$ 中去掉第 i 行、第 j 列后，剩下的余子矩阵 $\mathbf{A}_{ij}$ 的行列式
- 注意：余子矩阵 $\mathbf{A}_{ij}$ 是一个矩阵，余子式 M_{ij} 是一个标量——两者不要混淆

图1 · 3×3 方阵第1行各元素的余子矩阵

- (a)(b)(c)分别是 a_{11} 、 a_{12} 、 a_{13} 对应的余子矩阵：蓝色为当前元素，灰色为删去的行列，绿色为保留下来的余子矩阵
- $M_{11} = a_{22}a_{33} - a_{23}a_{32}$; $M_{12} = a_{21}a_{33} - a_{23}a_{31}$; $M_{13} = a_{21}a_{32} - a_{22}a_{31}$

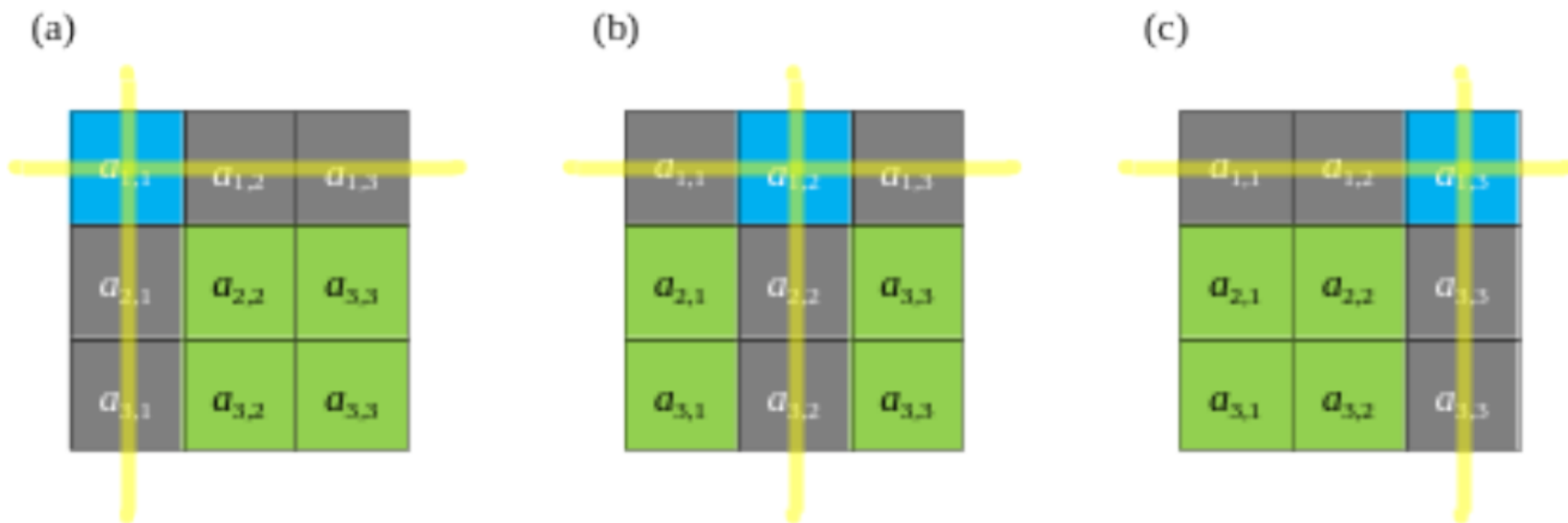


图 1. 3×3 方阵第 1 行各个元素的余子矩阵

图2 · 3×3 方阵第2行各元素的余子矩阵

- 我们可以选择沿不同的行、列展开——图2展示按第2行展开时，各元素对应的余子矩阵
- $M_{21} = a_{12}a_{33} - a_{13}a_{32}$; $M_{22} = a_{11}a_{33} - a_{13}a_{31}$; $M_{23} = a_{11}a_{32} - a_{12}a_{31}$

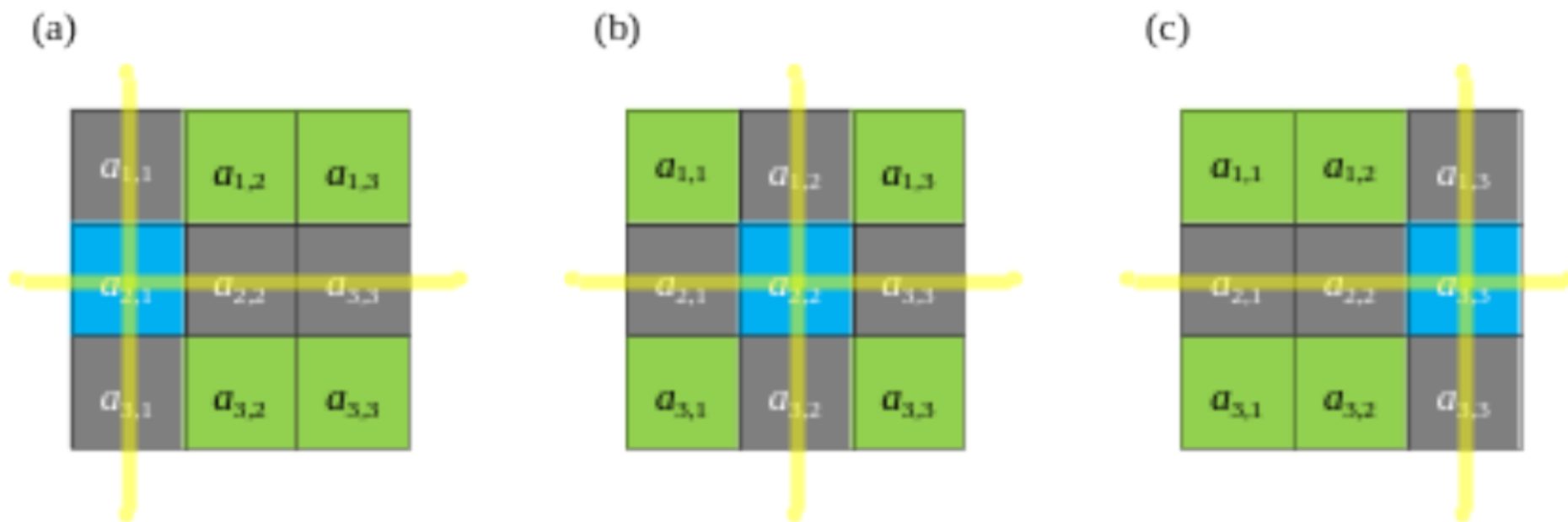


图 2. 3×3 方阵第 2 行各个元素的余子矩阵

图3 · 3×3 方阵第3行各元素的余子矩阵

- 图3展示按第3行展开后各元素的余子矩阵，请大家仿照图1、图2自行计算这三个余子式
- 方法完全一致：只是这一次去掉的是第3行，剩下的绿色部分构成余子矩阵

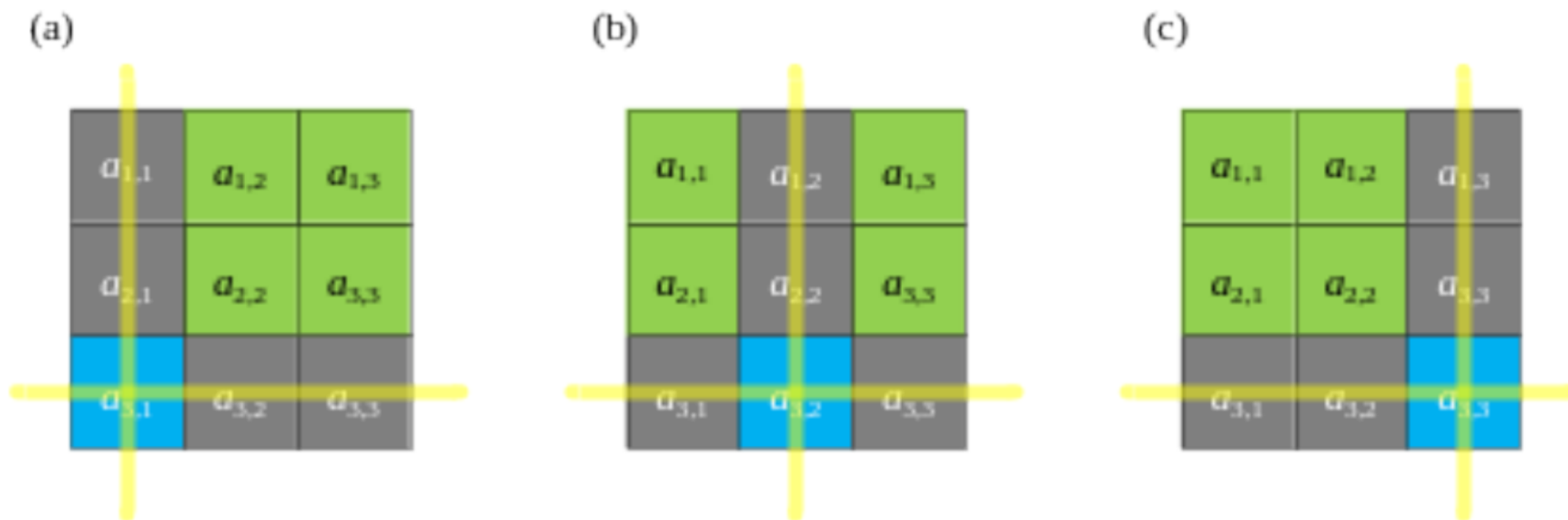


图 3. 3×3 方阵第 3 行各个元素的余子矩阵

给定一个具体的 3×3 方阵

$$A = [1 \ 2 \ 3; 4 \ 5 \ 6; 7 \ 8 \ 9]$$

- $M_{11} = |5 \ 6; 8 \ 9| = (5 \times 9) - (6 \times 8) = 45 - 48 = -3$
- $M_{12} = |4 \ 6; 7 \ 9| = (4 \times 9) - (6 \times 7) = 36 - 42 = -6$
- $M_{13} = |4 \ 5; 7 \ 8| = (4 \times 8) - (5 \times 7) = 32 - 35 = -3$
- 请大家仿照上面的过程，计算这个方阵其余六个余子式

代数余子式：给余子式加上符号

$$C_{ij} = (-1)^{i+j} M_{ij}$$

- 代数余子式是在余子式的基础上引入符号调整， $(-1)^{i+j}$ 是根据元素位置调整符号的因子
- 以上一页给出的方阵为例： $C_{11} = (-1)^{1+1} M_{11} = 1 \times (-3) = -3$ ； $C_{12} = (-1)^{1+2} M_{12} = -1 \times (-6) = 6$ ； $C_{13} = (-1)^{1+3} M_{13} = 1 \times (-3) = -3$
- 由于 $(-1)^{i+j}$ 形成交替正负的模式， 3×3 方阵的代数余子式符号调整矩阵为 $[+ \ - \ +; - \ + \ -; + \ - \ +]$

拉普拉斯展开：任选一行或一列求和

$$\det(A) = \sum_{j=1}^n a_{ij} C_{ij}$$

- 拉普拉斯展开提供了一种更具系统性的方法来计算行列式，可以沿任意一行、一列展开
- 对上例 3×3 方阵沿第1行展开： $\det(\mathbf{A}) = a_{11}C_{11} + a_{12}C_{12} + a_{13}C_{13} = 1 \times (-3) + 2 \times 6 + 3 \times (-3) = -3 + 12 - 9 = 0$
- 这个例子中三行 $[1,2,3]$ 、 $[4,5,6]$ 、 $[7,8,9]$ 彼此线性相关（等差数列），行列式恰好为0——这正呼应了4.3节“列（行）向量线性相关 $\Rightarrow$ 行列式为0”的结论

手解行列式的4个具体步骤

- a) 选取计算行列式展开时用到的某一行或列，提取余子矩阵（方阵）
- b) 根据余子矩阵（方阵），计算余子式（标量）
- c) 计算每个元素的代数余子式（标量），即调整符号后的余子式
- d) 根据拉普拉斯展开，将选定行或列的元素与其代数余子式相乘并求和，得到行列式的值——这种方法可以推广到更高维的矩阵

给定 3×3 方阵A，沿行、列方向逐步展开

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 0 & 1 \\ 1 & 2 & 1 \end{bmatrix}$$

- 接下来我们根据前面总结的4个步骤，对这个具体的 3×3 方阵，沿不同的行、列展开，一步步计算行列式
- 我们会发现：无论选择哪一行或哪一列展开，最终都会得到相同的行列式值

A) B) C) 逐项计算

9个元素的余子矩阵、余子式与代数余子式

元素	余子矩阵 $A_{i,j}$	余子式 $M_{i,j}$	代数余子式 $C_{i,j}$
$a_{1,1}$	$\begin{bmatrix} 0 & 1; 2 & 1 \end{bmatrix}$	-2	-2
$a_{1,2}$	$\begin{bmatrix} 3 & 1; 1 & 1 \end{bmatrix}$	2	-2
$a_{1,3}$	$\begin{bmatrix} 3 & 0; 1 & 2 \end{bmatrix}$	6	6
$a_{2,1}$	$\begin{bmatrix} 2 & 3; 2 & 1 \end{bmatrix}$	-4	4
$a_{2,2}$	$\begin{bmatrix} 1 & 3; 1 & 1 \end{bmatrix}$	-2	-2
$a_{2,3}$	$\begin{bmatrix} 1 & 2; 1 & 2 \end{bmatrix}$	0	0
$a_{3,1}$	$\begin{bmatrix} 2 & 3; 0 & 1 \end{bmatrix}$	2	2
$a_{3,2}$	$\begin{bmatrix} 1 & 3; 3 & 1 \end{bmatrix}$	-8	8
$a_{3,3}$	$\begin{bmatrix} 1 & 2; 3 & 0 \end{bmatrix}$	-6	-6

$A_{1,1}$ (去掉第1行、第1列) 为矩阵 $\begin{bmatrix} 0 & 1; 2 & 1 \end{bmatrix}$; 其余8个余子矩阵依此类推, 均为去掉对应行、列后剩下的 2×2 矩阵。

6条“平行”路径，结果都是 $\det(A) = 12$

展开方式	拉普拉斯展开公式	代入代数余子式	结果
第1行展开	$a_{1,1}C_{1,1}+a_{1,2}C_{1,2}+a_{1,3}C_{1,3}$	$1\times(-2)+2\times(-2)+3\times6$	12
第2行展开	$a_{2,1}C_{2,1}+a_{2,2}C_{2,2}+a_{2,3}C_{2,3}$	$3\times4+0\times(-2)+1\times0$	12
第3行展开	$a_{3,1}C_{3,1}+a_{3,2}C_{3,2}+a_{3,3}C_{3,3}$	$1\times2+2\times8+1\times(-6)$	12
第1列展开	$a_{1,1}C_{1,1}+a_{2,1}C_{2,1}+a_{3,1}C_{3,1}$	$1\times(-2)+3\times4+1\times2$	12
第2列展开	$a_{1,2}C_{1,2}+a_{2,2}C_{2,2}+a_{3,2}C_{3,2}$	$2\times(-2)+0\times(-2)+2\times8$	12
第3列展开	$a_{1,3}C_{1,3}+a_{2,3}C_{2,3}+a_{3,3}C_{3,3}$	$3\times6+1\times0+1\times(-6)$	12

无论沿哪一行、哪一列展开，都会得到 $\det(A) = 12$ ——这正是拉普拉斯展开定理保证的结论。

优先选择含零最多的行或列

$A = [1 \ 2 \ 3; 3 \ 0 \ 1; 1 \ 0 \ 1] \Rightarrow$ 沿第2列展开, 只需计算一个 2×2 行列式

- 在进行拉普拉斯展开时, 应优先选择包含最多0的行或列, 因为0元素对应的项不需要计算, 计算量会明显变小
- 如上例, 第2列为 $[2, 0, 0]^T$, 沿第2列展开时只有 a_{12} 这一项非零, 只需计算一个 2×2 余子式
- 如果遇到对角方阵、上三角矩阵、下三角矩阵, 行列式就更简单了——直接等于对角线元素之积 (详见4.1、4.2节)

用Python递归实现拉普拉斯展开

$$\text{determinant}(A) = \sum_{col} a[0,col] \times \text{cofactor}(0, col)$$

- 代码1展示如何用Python编程计算行列式：核心是两个函数——cal_minor()用来提取余子矩阵，determinant()用递归的方式实现拉普拉斯展开
- 递归的关键在于：determinant()函数在计算代数余子式时，会调用它自己，不断把大矩阵拆成小矩阵，直到 2×2 或 1×1 时才停止

递归计算行列式

LA_04_04_01.ipynb

- a) 定义cal_minor(A, row_idx, col_idx)函数，用于提取余子矩阵
- b) 用numpy.delete()先删掉指定行 (axis=0)，再删掉指定列 (axis=1)，得到余子矩阵
- c) 定义determinant(A)函数，目标是计算方阵A的行列式
- d) 递归终止条件：1×1矩阵直接返回该数字；2×2矩阵按对角线相乘再相减
- e) 沿第一行展开：循环每一列，调用cal_minor取得余子矩阵，递归调用determinant()算出代数余子式，用det += A[0, col] * cofactor完成拉普拉斯展开

```
## 初始化
import numpy as np

## 自定义函数，提取余子矩阵
def cal_minor(A, row_idx, col_idx):
    A_i = np.delete(A, row_idx, axis=0)
    A_ij = np.delete(A_i, col_idx, axis=1)
    # 去掉指定的行、列
    return A_ij

## 用Laplace展开递归计算行列式
def determinant(A):
    n = A.shape[0]
    if n == 1:
        return A[0, 0]
    if n == 2:
        return A[0, 0] * A[1, 1] - A[0, 1] * A[1, 0]

    det = 0
    for col_idx in range(n):
        minor = cal_minor(A, 0, col_idx) # 沿第一行展开
        cofactor = ((-1) ** col_idx) * determinant(minor)
        # 计算代数余子式
        # 相当于 (-1) ** ((col_idx + 1) + 1) = (-1) ** col_idx
        det += A[0, col_idx] * cofactor # 计算行列式

    return det

## 测试
A = np.array([[1, 2, 3],
              [3, 0, 1],
              [1, 2, 1]])
A_det = determinant(A)
```

代码 1. 计算行列式 | LA_04_04_01.ipynb

余子式 → 代数余子式 → 拉普拉斯展开

$$M_{ij} = \det(A_{ij}) \Rightarrow C_{ij} = (-1)^{i+j} M_{ij} \Rightarrow \det(A) = \sum_j a_{ij} C_{ij}$$

- 从余子矩阵到余子式，再到代数余子式，最后通过拉普拉斯展开求和——这4步构成了手解行列式的完整流程，且可以推广到任意 $n \times n$ 方阵
- 本节的递归Python实现，正是这套流程最自然的程序化表达；实际工程计算中，我们通常直接调用`numpy.linalg.det()`，但理解这套手算流程有助于建立对行列式的直觉

本节关键术语

余子矩阵 *Minor Matrix $A_{i,j}$* —— 去掉第*i*行、第*j*列后剩下的 $(n-1) \times (n-1)$ 矩阵

代数余子式 *Cofactor $C_{i,j}$* —— $C_{i,j} = (-1)^{(i+j)} M_{i,j}$, 符号调整后的余子式

符号调整因子 $(-1)^{(i+j)}$ —— 按元素位置形成交替正负的棋盘状模式

列展开 $\det(A) = \sum_i a_{i,j} C_{i,j}$, 固定列*j*, 对行*i*求和

对角线法则 *Sarrus' Rule* —— 仅适用于 3×3 矩阵的行列式速算法, 见本节练习Q4

余子式 *Minor $M_{i,j}$* —— 余子矩阵 $A_{i,j}$ 的行列式, 是一个标量

拉普拉斯展开 *Laplace Expansion* —— 沿任意一行或列, 元素乘代数余子式并求和得到行列式

行展开 $\det(A) = \sum_j a_{i,j} C_{i,j}$, 固定行*i*, 对列*j*求和

递归算法 *determinant()* 函数调用自身, 将 $n \times n$ 行列式归约为 2×2 、 1×1 的基本情形

零元素优先 沿含零最多的行或列展开, 可以跳过对应的乘法运算, 减少计算量

配套代码与延伸阅读

延伸阅读 • `numpy.linalg.det`

<https://numpy.org/doc/stable/reference/generated/numpy.linalg.det.html> —— 直接计算行列式的NumPy函数

延伸阅读 • `numpy.delete`

<https://numpy.org/doc/stable/reference/generated/numpy.delete.html> —— 代码1中用于提取余子矩阵的核心函数

延伸阅读 • 对角线法则 (Sarrus' Rule)

仅适用于 3×3 矩阵的行列式速算技巧，是拉普拉斯展开的一个特例，见练习Q4

开源仓库

全书代码与图像素材: <https://github.com/Visualize-ML>

习题 Q1 – Q9

Q1 对以下三个方阵，各自任选一行或一列，计算对应元素的余子式（标量），并思考哪一行、哪一列展开最方便：
 $A = [1, 0, 0; 1, 1, 1; 1, 1, 1]$; $B = [1, 1, 1; 0, 1, 0; 0, 1, 1]$; $C = [1, 1, 1; 1, 1, 1; 0, 0, 1]$

Q3 请大家用列方向展开，计算矩阵 $A = [1, 2, 3; 3, 0, 1; 1, 2, 1]$ 的行列式，验证结果是否仍为12。

Q5 请修改代码1，增加语句判断输入矩阵是否为方阵，只有方阵才继续计算行列式。

Q7 写一个函数 `verify_laplace(A)`，对 3×3 方阵分别沿每一行、每一列展开计算行列式，验证6个结果是否都相等。

Q9 写一个函数 `all_minors(A)`，返回 $n \times n$ 方阵 A 所有元素对应余子式组成的 $n \times n$ 数组。

Q2 用本节拉普拉斯展开公式，计算如下 2×2 矩阵的行列式： $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 。

Q4 请自学对角线法则（Sarrus法则），并说明它与本节拉普拉斯展开之间的关系。

Q6 请修改代码1，增加语句优先选择含零最多的行或列展开，以减少计算量。

Q8 用 `numpy.linalg.det()` 验证本节完整例子中矩阵 $A = [1, 2, 3; 3, 0, 1; 1, 2, 1]$ 的行列式是否等于12。

习题 Q10 – Q18

Q10 写一个函数`all_cofactors(A)`，在`all_minors`的基础上添加符号，返回所有代数余子式组成的矩阵。

Q12 写一个函数`laplace_expand(A, row_idx)`，指定按某一行展开计算行列式，并与`numpy.linalg.det()`的结果比较。

Q14 生成10个随机 4×4 矩阵，分别用扩展到 4×4 的递归Laplace展开和`numpy.linalg.det()`计算行列式，比较结果与耗时。

Q16 写一个函数`count_zeros(A)`，统计方阵A每一行、每一列中零元素的个数，返回零元素最多的行或列索引。

Q18 测试：当矩阵维度从 3×3 增加到 8×8 时，递归Laplace展开的计算耗时如何随维度增长？画出耗时-维度关系图。

Q11 探索：代数余子式矩阵的转置与原矩阵A有什么关系？（提示：可自学“伴随矩阵”`adj(A)`与行列式的关系）

Q13 写一个函数`laplace_expand_col(A, col_idx)`，指定按某一列展开计算行列式，并与`numpy.linalg.det()`的结果比较。

Q15 探索：代码1中的`determinant`函数目前只沿第一行展开，请修改为可以指定任意一行展开的版本。

Q17 结合Q16，写一个函数`smart_determinant(A)`，优先沿零元素最多的行或列展开计算行列式。

习题 Q19 – Q26

Q19 探索：为什么说拉普拉斯展开的计算复杂度是阶乘级别 $O(n!)$ ，而后续章节将介绍的高斯消元法只需 $O(n^3)$ ？

Q21 探索：如果方阵A某一行是另一行的k倍（线性相关），用拉普拉斯展开计算行列式，请用具体例子验证结果为0。

Q23 探索：用sympy库对含符号变量的 3×3 矩阵 $[[a,b,c],[d,e,f],[g,h,i]]$ 做符号化拉普拉斯展开，输出完整代数表达式。

Q25 给定一个上三角矩阵，分别用本节拉普拉斯展开和4.2节对角线乘积公式计算行列式，验证结果一致并比较计算量。

Q20 写一个函数`print_all_minors(A)`，构造并打印一个 3×3 矩阵所有9个余子矩阵，用于教学展示。

Q22 写一个函数`sarrus_3x3(A)`，实现对角线法则（Sarrus法则）计算 3×3 矩阵的行列式，并与本节余子式展开法比较结果。

Q24 写一个函数`compare_expansion_speed(A)`，分别沿零元素最少和最多的行/列展开，统计各自的乘法运算次数。

Q26 结合本节所学，写一份简短总结（可以是代码注释形式），说明余子式、代数余子式、拉普拉斯展开三个概念之间的递进关系。

10 个可深入探讨的 Prompt (1/2)

- 1 请解释余子式 (minor) 和余子矩阵 (minor matrix) 的区别, 为什么说前者是标量、后者是矩阵?
- 2 请解释代数余子式为什么要在余子式基础上乘以 $(-1)^{i+j}$, 这个符号规律背后的含义是什么?
- 3 请解释拉普拉斯展开的一般公式, 以及为什么沿任意一行或一列展开都能得到相同的行列式值?
- 4 请解释为什么优先选择含零最多的行或列展开可以减少计算量, 具体节省了哪些步骤?
- 5 请解释拉普拉斯展开与递归算法之间的关系, 为什么计算 $n \times n$ 矩阵的行列式最终会归约到 2×2 或 1×1 ?

10 个可深入探讨的 Prompt (2/2)

6 请解释代码1中determinant函数的递归终止条件，如果去掉 2×2 的特殊处理，代码是否依然正确？

7 请解释拉普拉斯展开的计算复杂度为什么是 $O(n!)$ ，这对大规模矩阵的行列式计算有什么实际影响？

8 请解释对角线法则（Sarrus法则）的适用范围，以及为什么它不能直接推广到 4×4 或更高维矩阵？

9 请结合本节的完整例子（6条路径都得到 $\det(A)=12$ ），谈谈拉普拉斯展开体现了行列式的哪些重要性质？

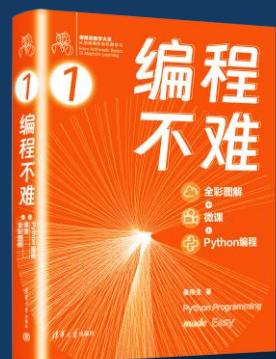
10 请结合本章（4.1-4.4节）所学，总结几何法（面积/体积）和代数法（拉普拉斯展开）计算行列式各自的优势与适用场景。

数学

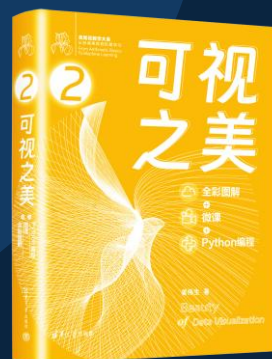
数学基础

线性代数

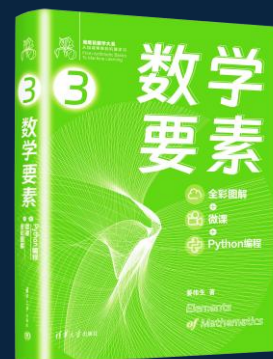
概率统计



Python编程



数据可视化



数据分析
图和网络

回归、降维
分类、聚类

工具

实践

